2023

1. 10 pts) Permutation generator

Assume it takes O(1) to swap two elements (i.e., swap(a[i], a[j])).

```
void Permutations(char *a, const int k, const int m)
// Generate all the permutations of a[k], ..., a[m]
{
    if (k == m) { // output permutation
        for (int i = 0; i <= m; i++) cout << a[i] << " ";
        cout << endl;
    }
    else { // a[k:m] has more than one permutation. Generate these
recursively.
        for (i = k; i <= m; i++) {
            swap(a[k], a[i]);
            Permutations(a, k+1, m);
            swap(a[k], a[i]);
        }
    }
}
```

a. (5 pts) Please insert a new variable, count, into the program and show a simplified version of the original program and show the time complexity according to it.

b. (5 pts) Please build a table which lists the total number of steps contributed by each statement as follows and show the time complexity according to it.

欄位為：line / s/e / frequency / total steps

line	s/e	frequency	total steps
1			
2			
⋮			
13			
		Total number of steps	

2. 題目中文翻譯

1.（10 分）排列產生器
假設交換兩個元素所需時間為 O(1)，也就是 swap(a[i], a[j]) 可視為常數時間。

給定以下遞迴程式 Permutations(char *a, const int k, const int m)，它會產生陣列中 a[k] 到 a[m] 的所有排列。

a.（5 分）請在程式中加入一個新變數 count，寫出原程式的簡化版本，並根據這個 count 推導其時間複雜度。
b.（5 分）請建立一個表格，列出每一行敘述所貢獻的總步數，表格格式如：line / s/e / frequency / total steps，並根據此表說明時間複雜度。

3. 詳細解答

這題的核心是分析排列產生程式的時間複雜度。因為這個函式會列出所有排列，而長度為 (n) 的資料共有 (n!) 種排列，所以時間複雜度一定至少和 (n!) 有關。若令子問題規模為
n=m−k+1

表示目前還有 (n) 個元素要排列。

先看遞迴行為：當 (k=m) 時，代表只剩最後一個位置，這時輸出整個排列。輸出時 for (int i=0; i<=m; i++) 需要印出 (m+1) 個元素，若把整體長度記成 (n)，則一次輸出成本為 (O(n))。而這樣的葉節點共有 (n!) 個，因為每一個葉節點對應一個完整排列，所以光輸出就要花
O(n*n!)

若依題目要求加入 count，最簡化的版本可寫成只計算遞迴次數，不真的輸出：

```
int count = 0;

void Permutations(char *a, const int k, const int m) {
    count++;    // 每次呼叫函式就記一步
    if (k == m) return;
    for (int i = k; i <= m; i++) {
        swap(a[k], a[i]);
        Permutations(a, k + 1, m);
        swap(a[k], a[i]);
```

```
    }
}
```

若令 (T(n)) 表示還有 (n) 個元素要排列時的執行時間，則遞迴式可寫成

T(1)=c

T(n)=nT(n-1)+cn

原因是：當還有 (n) 個位置可以選時，for 迴圈會跑 (n) 次；每次都會遞迴呼叫一次 Permutations(..., k+1, m)，所以有 nT(n-1)；另外每次迴圈中還有兩次 swap 與迴圈控制等常數操作，合起來是 cn。

若題目把「輸出 permutation」也算進去，則 base case 不只是常數，而是 (O(n))。因此更完整的遞迴式應寫成

T(1)=O(n)

T(n)=nT(n-1)+O(n)

展開後可得整體為
T(n)=O(n*n!)

這就是此排列產生器的正確時間複雜度。很多同學會只寫 (O(n!))，但這少算了每次輸出排列時要印出 (n) 個元素的成本，所以更精確答案應是
T(n)=O(n*n!)

若題目第 b 小題要做步數表，我們可先令總長度為 (n=m+1)，並簡化各行的單步成本如下：每一行敘述執行一次的成本皆視為常數 c_i。則重點行數分析如下。

line 2：`if (k == m)`
每次函式呼叫都要判斷一次，所以其頻率等於總呼叫次數。

line 3：`for (int i = 0; i <= m; i++) cout << a[i] << " ";`
只在葉節點執行。葉節點數為 n!，而每次要跑 n 次，因此頻率約為 n·n!。line

4：cout << endl;
每個排列輸出一次，所以頻率為 n!。

line 7：for (i = k; i <= m; i++)
在規模為 (r) 的每個遞迴層，迴圈跑 r 次。

line 8：swap(a[k], a[i]);
每進入一次迴圈做一次。

line 9：Permutations(a, k+1, m);
每進入一次迴圈呼叫一次下一層遞迴。

line 10：swap(a[k], a[i]);
每次遞迴返回後再交換回去一次。

若把所有層加總，交換與遞迴呼叫的總次數都和 n! 同階，而葉節點輸出的總成本是最大的主導項之一，因此最後總步數仍然由
O(n*n!)
主導。

所以本題最後結論可寫成：

加入 count 後，遞迴關係式為

T(n)=nT(n-1)+O(n)
其中 base case 需要輸出一整個排列，因此總時間複雜度為

O(n·n!)

若老師要你在考卷上寫簡潔版，可以直接寫：

設 (n=m-k+1)。此程式對每個位置都嘗試與後面所有元素交換，因此遞迴式為

T(n)=nT(n-1)+cn

且共有 (n!) 個葉節點。每個葉節點要輸出整個排列，共 (n) 個元素，因此輸出成本為 O(n·n!)。故排列產生器整體時間複雜度為
[
O(n·n!)

]
若不計輸出成本，只看遞迴與交換，則可近似為
O(n!)
但完整精確答案應為
O(n⋅n!)

2. 詳細解答（可直接抄寫）
   設 n=m−k+1n = m - k + 1n=m−k+1。此演算法利用遞迴與交換產生所有排列，每一層會將第 k 個位置與後面 n 個元素逐一交換，因此遞迴關係為

T(n)=nT(n−1)+cnT(n) = nT(n-1) + cnT(n)=nT(n−1)+cn

其中 cncncn 為 for 迴圈與 swap 的成本。當 k=mk = mk=m 時，需輸出整個排列，成本為 O(n)O(n)O(n)。

因為共有 n!n!n! 個排列，每個排列需輸出 n 個元素，所以輸出總成本為

O(n⋅n!)O(n \cdot n!)O(n⋅n!)

因此整體時間複雜度為

O(n⋅n!)\boxed{O(n \cdot n!)}O(n⋅n!)

加入 count 的簡化版本：

```
int count = 0;

void Permutations(char *a, const int k, const int m) {
    count++;
    if (k == m) return;
    for (int i = k; i <= m; i++) {
        swap(a[k], a[i]);
        Permutations(a, k+1, m);
        swap(a[k], a[i]);
    }
}
```

步數分析表（重點寫法）

line 1：c1，frequency ≈ n!，total ≈ c1·n!
line 2：c2，frequency = n!，total = c2·n!
line 3：c3，frequency = n·n!，total = c3·n·n!
line 4：c4，frequency = n!，total = c4·n!
line 7：c5，frequency ≈ n·n!，total = c5·n·n!
line 8：c6，frequency ≈ n·n!，total = c6·n·n!
line 9：c7，frequency ≈ n·n!，total = c7·n·n!
line10：c8，frequency ≈ n·n!，total = c8·n·n!

總步數

$T(n)=O(n \cdot n!)$

結論（一句話）
此演算法產生 n! 個排列且每次輸出需 n 步，因此時間複雜度為 $O(n \cdot n!)$

二、(10 pts) The class Bag is a container which is stored by an integer array. Please modify the following code using the Template.

```
class Bag {
public:
    Bag(int bagCapacity = 10);
    ~Bag();
    void Pop();

private:
    int *array;
    int capacity; // capacity of array
    int top; // array position of top element
};
Bag::Bag(int bagCapacity) : capacity(bagCapacity) {
    if (capacity < 1) throw "Capacity must be > 0";
    array = new int[capacity];
    top = -1;
}

Bag::~Bag() { delete [] array; }

void Bag::Pop() {
    if (IsEmpty()) throw "Bag empty, cannot delete";
    int deletePos = top / 2;
    copy(array + deletePos + 1, array + top + 1, array + deletePos);
    top--;
}
```

2. 題目中文翻譯

   2.（10 分）Bag 類別是一種以整數陣列實作的容器。請使用 C++ Template（模板）將以下程式碼修改，使其能支援不同資料型態，而不僅限於 int。

3. 詳細解答

   本題的重點是將原本只能存放 int 的 Bag 類別，改寫為「泛型（Generic）」結構，也就是使用 C++ Template，使這個容器可以存放任何資料型別（例如 int、double、char、甚至自訂類別）。這樣的設計符合資料結構中 ADT（抽象資料型態）的精神，讓資料與操作分離，提高程式重用性與彈性。

首先，需在 class 前加入 template <class T> 或 template <typename T>，表示此類別使用型別參數 T。接著將所有原本的 int 改為 T，例如 int *array 改為 T *array，代表這個陣列可以儲存任意型別。建構子中的 new int[capacity] 也要改為 new T[capacity]，確保記憶體配置符合泛型型別。

另外，在 Pop() 函式中，copy() 操作是將陣列中間元素往前移動來填補刪除的位置，這部分對任何型別都成立，因此不需特別修改邏輯，只要型別一致即可。唯一要注意的是 template 類別的成員函式定義時，也必須加上 template <class T> 並使用 Bag<T>:: 的形式。

完整修改如下：

```
template <class T>
class Bag {
public:
    Bag(int bagCapacity = 10);
    ~Bag();
    void Pop();
    bool IsEmpty() const { return top == -1; }

private:
    T *array;
    int capacity;
    int top;
};

template <class T>
Bag<T>::Bag(int bagCapacity) : capacity(bagCapacity) {
    if (capacity < 1) throw "Capacity must be > 0";
    array = new T[capacity];
    top = -1;
}

template <class T>
Bag<T>::~Bag() {
    delete [] array;
}

template <class T>
```

```
void Bag<T>::Pop() {
    if (IsEmpty()) throw "Bag empty, cannot delete";
    int deletePos = top / 2;
    copy(array + deletePos + 1, array + top + 1, array + deletePos);
    top--;
}
```

透過上述修改，Bag 不再限制只能存放整數，而可以用 Bag<int>、Bag<double>、Bag<char> 等方式建立不同型別的容器。這樣的設計提升了程式的彈性與重用性，是 C++ 中 Template 的典型應用。結論是：Template 的目的在於將資料型別抽象化，使同一套邏輯可以套用在不同型別上，是實作泛型資料結構的重要工具。

第 3 題 3.Essay Questions (30 pts)

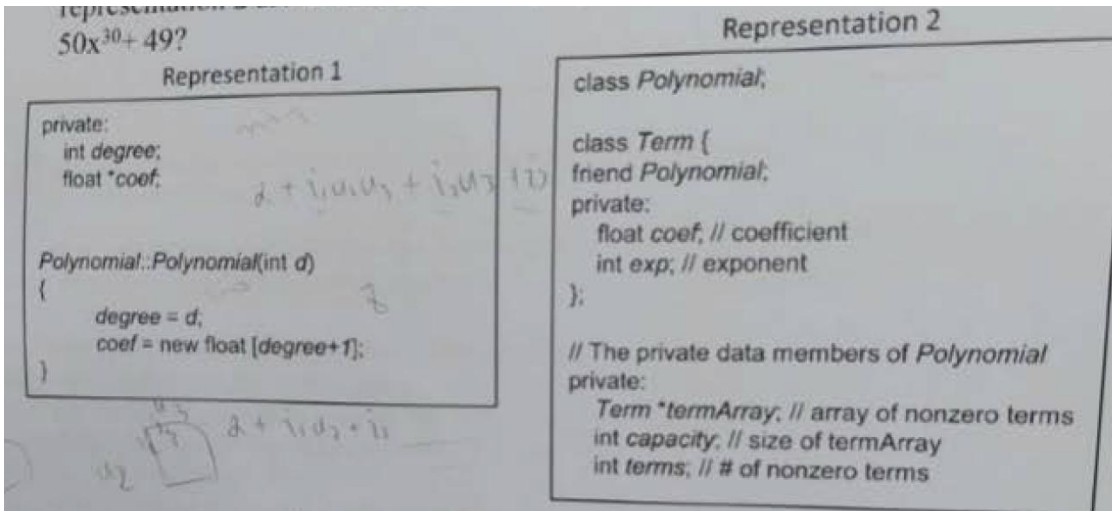
a. (3 pts) When you estimate the time complexity of an algorithm, what constitutes a step is defined after which property has been determined?

b. (3 pts) In addition to 1) support of objects and 2) requirement of objects belong to a class, what is the third property to constitute an object-oriented language?

c. (3 pts) Please show the location when you define class ChainIterator with respect to the class Chain which is singly linked list?

d. (3 pts) How many space is required when a representation 1 and a representation 2 are used if we have a polynomial equation

$f(x) = 3x^{100} + 4x^{98} + 50x^{30} + 49$?



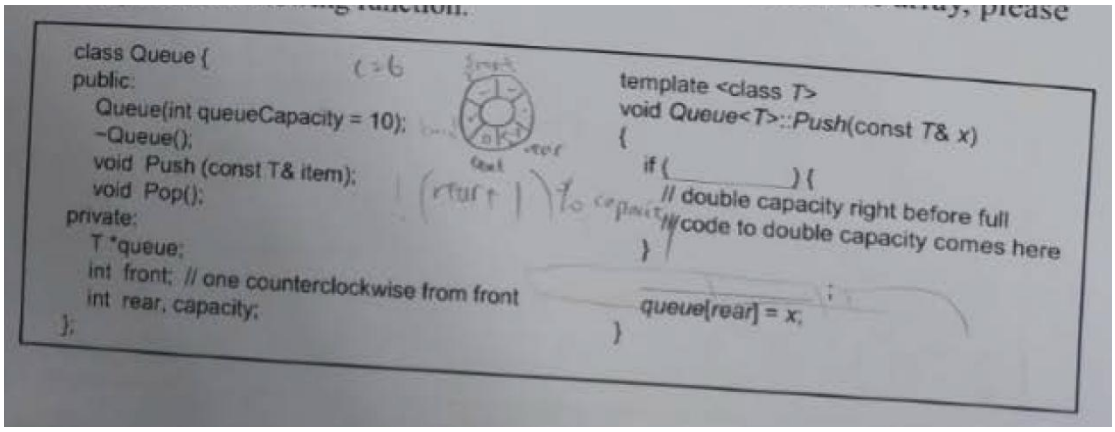
e. (3 pts) Given a sparse matrix which is represented by a circular list, how many nodes do you have if it contains (r) nonzero elements?

f. (3 pts) Please show the equation to represent the address of a C++ array $a[i_1][i_2]\cdots[i_n]$ if it starts at the address α
(note that it is stored in the row major) and the dimension of the array is $a[u_1][u_2]\cdots[u_n]$.

g. (3 pts) What are the nested classes? Compared to decomposed classes, what is the problem with the nested classes?

h. (3 pts) Given a circular queue which is stored by an array, where front and rear are the pointers for insertion and deletion of an element from the array, please complete the

following function:



```
class Queue {
public:
    Queue(int queueCapacity = 10);
    ~Queue();
    void  Push (const T& item);
    void  Pop();
private:
    T *queue;
    int  front;  // one counterclockwise from front
    int  rear, capacity;
};
```

```
template <class T>
void Queue<T>::Push(const T& x)
{
    if (_____) {
        // double capacity right before full
        // code to double capacity comes here
    }
    _____;
    queue[rear] = x;
}
```

```
template <class T>
class Queue {
    static ChainNode<T> *av; // available space list
public:
    // public members here
private:
    // other private members here
};
```

i. (3 pts) Compared to the singly linked list, what is the most important advantage of the doubly linked list?

j. (3 pts) Please explain the purpose of setting the available list av as static data member of the class CircularList as follows:

```
template <class T>
class CircularList {
    static ChainNode<T> *av; // available space list
public:
    // public members here
private:
    // other private members here
};
```

2. 題目中文翻譯
3. 問答題（30 分）

a.（3 分）當你估計一個演算法的時間複雜度時，「一步（step）」要如何定義？它是在先確定哪一項性質之後才決定的？

b.（3 分）除了 1）支援物件、2）要求物件必須屬於某個類別之外，構成物件導向語言的第三個性質是什麼？

c.（3 分）若要定義單向鏈結串列 Chain 的 ChainIterator 類別，請說明 ChainIterator 應該定義在相對於 Chain 類別的什麼位置。

d.（3 分）若多項式

$f(x) = 3x^{100} + 4x^{98} + 50x^{30} + 49$？

分別使用 representation 1 與 representation 2 來表示，請問各需要多少空間？

e.（3 分）若一個稀疏矩陣以 circular list（循環串列）表示，且它含有 (r) 個非零元素，則總共有多少個節點？

f.（3 分）若 C++ 陣列

f. (3 pts) Please show the equation to represent the address of a C++ array $a[i_1][i_2]\cdots[i_n]$ if it starts at the address α
(note that it is stored in the row major) and the dimension of the array is $a[u_1][u_2]\cdots[u_n]$.

請寫出其位址表示公式。

g.（3 分）什麼是 nested classes（巢狀類別）？和 decomposed classes（分解式類別）相比，nested classes 的問題是什麼？

h.（3 分）已知 circular queue 以陣列實作，front 與 rear 為刪除與插入的位置指標，請補完 Push 函式。

i.（3 分）與 singly linked list（單向鏈結串列）相比，doubly linked list（雙向鏈結串列）最重要的優點是什麼？

j.（3 分）請說明為什麼在 CircularList 類別中，會把 available list av 設成 static data member。

3. 詳細解答

a. 估計演算法時間複雜度時，「一步 step」的定義，必須先建立在「所使用的計算模型或基本運算單位」已經被決定之後，才能合理定義。也就是說，要先

確定哪些操作可以視為基本操作，例如指定、比較、加減乘除、陣列存取、指標移動等，然後才把它們算成一步。若沒有先決定這個標準，同一個演算法可能會因為不同人採用不同基本單位而得到不同分析結果。因此這裡所問的性質，通常就是先確定「基本操作的成本模型是否視為常數時間」，也可寫成先確定 machine model 或 computational model，再定義 what constitutes a step。簡單講，就是先決定「哪一種基本運算算一步」，再做時間分析。

b. 物件導向語言的三個關鍵性質中，題目已給前兩項：第一是支援 objects，第二是 objects 必須屬於 classes。第三項就是支援「繼承 inheritance」。也就是說，一個類別可以從另一個類別延伸出來，繼承其資料成員與成員函式，再進一步擴充或修改。這是物件導向語言很重要的精神，因為它能促進程式碼重用、建立階層式設計，也讓多型與抽象化更容易實作。所以第 b 小題最標準的答案就是：the third property is inheritance。

c. ChainIterator 是用來走訪 Chain 的迭代器類別。若以課本常見寫法，ChainIterator 通常與 Chain 密切相關，因此會宣告成 Chain 的 friend class，並且可以定義在 Chain 類別之前，或作為與 Chain 平行的另一個類別，再由 Chain 宣告它為朋友類別。若題目強調「with respect to class Chain」，最常見答案是：ChainIterator 應定義在 Chain 類別外部，但要讓 Chain 宣告 friend class ChainIterator;，因為 iterator 需要存取串列節點或 first 節點等內部結構。也可以說它是和 Chain 分開定義，但在邏輯上緊鄰 Chain，通常放在 ChainNode 與 Chain 定義附近。

d. 題目給的多項式是

$f(x) = 3x^{100} + 4x^{98} + 50x^{30} + 49$?

共有 4 個非零項，最高次為 100。

Representation 1 的圖中表示方式是：
有一個 int degree，再加上一個 float *coef 指向大小為 degree+1 的係數陣列。
因此空間需求為：
一個 int degree
一個指標 float *coef
以及 degree+1 = 101 個 float

所以 representation 1 需要的空間可寫成：

$$1 \text{ int} + 1 \text{ pointer} + 101 \text{ floats}$$

Representation 2 是只

存非零項，每一項用一個 Term，而 Term 裡面有：
一個 float coef
一個 int exp
此外 Polynomial 本身還有：
一個 Term *termArray
一個 int capacity
一個 int terms

因為此多項式有 4 個非零項，所以 representation 2 需要：
4 個 Term，也就是 4×(1 float+1 int)再加上 1 個 Term*、2 個 int

所以可寫成：

$$1\ \text{pointer} + 2\ \text{ints} + 4\ \text{Terms}$$

其中

$$4\ \text{Terms} = 4\ \text{floats} + 4\ \text{ints}$$

若合併表示：
representation 2 需要
1 pointer+6 ints+4 floats
因為 4 個 Term 裡有 4 個 int，再加上 capacity 與 terms 共 2 個 int。

所以結論是：當多項式非常稀疏時，representation 2 比 representation 1 節省很多空間。因為 representation 1 必須開到 101 個係數位置，但 representation 2 只要存 4 個非零項。

e. 稀疏矩陣若以 circular list 表示，除了 (r) 個非零元素節點外，通常還需要 header nodes。依資料結構教材的標準表示法，若矩陣有 (p) 列、(q) 行，則需要一個主頭節點，再加上 max(p,q)個 header nodes，以及 (r) 個非零元素節點。不過題目只給 (r) 而沒有列數與行數，通常老師想考的是「至少有非零元素節點加上表頭節點」。若依最常見簡答寫法，可答：共有 (r+1) 個主要資料節點結構中的 element nodes 與 head node。若依完整 orthogonal/circular linked representation，則應寫：
r+max(rows,cols)+1
個節點。考場上若老師課堂講的是 sparse matrix circular list 標準表示法，建議寫完整版最安全。

f. 若 n 維陣列以 row-major 儲存，起始位址為 α，每個元素大小為 w，則

$$LOC(a[i_1][i_2]\cdots[i_n]) = \alpha + w\Big(((\cdots((i_1u_2+i_2)u_3+i_3)\cdots)u_n+i_n)\Big)$$

這是從每一維往後累積的 row-major 公式。

若展開寫成更完整形式：

$$LOC(a[i_1][i_2]\cdots[i_n]) = \alpha + w\left(i_1u_2u_3\cdots u_n + i_2u_3u_4\cdots u_n + \cdots + i_{n-1}u_n + i_n\right)$$

這就是 row-major order 的位址計算式。意思是前面維度變動得慢，最後一維變動得最快。

g. Nested classes 就是一個類別定義在另一個類別裡面，例如在 Polynomial 內定義 Term。這樣做的好處是可以強調該內部類別只服務於外部類別，封裝性較高，名稱也比較不容易和外界衝突。
但和 decomposed classes 相比，nested classes 的問題是：它的重用性較差、獨立性較低、模組化比較差。因為巢狀類別通常和外部類別綁得很緊，若其他地方也想使用這個類別，就不如拆開成獨立類別來得方便。換句話說，decomposed classes 比較適合獨立設計、測試與重複利用；nested classes 則較封閉，延展與重用較不方便。

h. Circular queue 用陣列實作時，判斷快滿的條件是

$$(rear+1)\%capacity == front$$

因為這代表再插入一個元素就會撞到 front。插入時 rear 要先往前移一格：

$$rear = (rear+1)\%capacity$$

所以完整填空為：

```
template <class T>
void Queue<T>::Push(const T& x)
{
    if ((rear + 1) % capacity == front) {
        // code to double capacity comes here
    }
    rear = (rear + 1) % capacity;
    queue[rear] = x;
}
```

i. 相較於 singly linked list，doubly linked list 最重要的優點是：它可以雙向走訪，也就是每個節點同時保有 llink 與 rlink（或 prev 與 next），因此不只

能往後走，也可以往前走。這使得刪除目前節點或在某節點前插入時更方便，因為可以直接找到前驅節點，不必像單向串列那樣常常需要從表頭重新搜尋前一個節點。因此雙向鏈結串列在需要頻繁刪除、反向走訪、雙端操作時特別有優勢。

j. 把 available list av 設為 static data member 的目的，是讓所有 CircularList<T> 物件共享同一個可用節點串列。也就是說，當某個 circular list 刪除節點時，這個節點可以放回共同的 available list；當另一個 circular list 需要新節點時，也可以直接從這個共同的 av 取出，而不必每次都向系統重新申請記憶體。這樣做有三個好處：第一，減少動態配置與釋放記憶體的次數，提高效率；第二，讓已釋放的節點能被不同物件重複利用，降低記憶體浪費；第三，因為 av 是 class level 而不是 object level，所以所有同型別的 CircularList<T> 都能共用同一個 free list。簡單一句話總結：av 設成 static，是為了讓所有 circular list 共享可回收節點空間，提升記憶體管理效率與重用性。

```
template <class T>
class CircularList {
   static ChainNode <T> *av; // available space list
public:
   // public members here
private:
   // other private members here
};
```

4. (10 pts) Give a table which characterizes the priority of the operators.
The expression (e) is

A&&B||C||!(E>F)

Note that you have to show the process of change in a stack when you reply the following questions.

a. (2 pts) Show the equivalent postfix expression if we use a stack to evaluate the expression.
b. (5 pts) Show the process to evaluate the value of (e) according to the result obtained in (a) where (A=True,; B=True,; C=False,; E=3,; F=4).
c. (3 pts) Show the equivalent prefix expression if we use a stack to evaluate the expression. Hint: you have to scan it from right to left.

運算子優先權表（題目所附）大意如下：
! 優先於 * / %，再來是 + -，再來是 < >，再來是 == !=，再來是 &&，最後是 ||，括號另外處理。

2. 題目中文翻譯
   4.（10 分）請根據運算子的優先順序表回答下列問題。
   已知運算式

   A&&B||C||!(E>F)

注意：回答下面問題時，必須顯示使用 stack（堆疊）時，堆疊內容變化的過程。

a.（2 分）若使用 stack 將此中序運算式轉成 postfix（後序式），請寫出其等價的 postfix expression。
b.（5 分）根據 a 小題得到的 postfix expression，在 (A=True,; B=True,; C=False,; E=3,; F=4) 的情況下，請用 stack 顯示整個求值過程，並求出 (e) 的值。
c.（3 分）若使用 stack 求其 prefix（前序式），請寫出等價的 prefix expression。提示：要由右往左掃描。

3. 詳細解答
   詳細解答
   這題要做三件事：先把中序式轉成 postfix，再用 postfix 求值，最後再轉成 prefix。原式是

A&&B||C||!(E>F)

根據優先順序，！ 最高，> 高於 &&，而 && 高於 ||。所以整個式子實際的結構可理解成

((A&&B)||C)||(!(E>F))

先做 a 小題，轉 postfix。
我們從左到右掃描：

讀到 A，是 operand，直接輸出。
輸出：A
stack：空

讀到 &&，push。
輸出：A
stack：&&

讀到 B，直接輸出。
輸出：AB
stack：&&

讀到 ||，因為 stack top 是 &&，而 && 優先權高於 ||，所以先把 && pop 出來輸出，再把 || push 進去。
輸出：AB&&
stack：||

讀到 C，直接輸出。
輸出：AB&&C
stack：||

讀到下一個 ||，此時 stack top 也是 ||，同優先層，故先 pop 再 push 新的 ||。
輸出：AB&&C||
stack：||

讀到 ！，！ 優先權比 || 高，因此直接 push。
輸出：AB&&C||
stack：||, !

讀到 (，直接 push。
輸出：AB&&C||
stack：||, !, (

讀到 E，直接輸出。
輸出：AB&&C||E
stack：||, !, (

讀到 >，push。
輸出：AB&&C||E
stack：||, !, (, >

讀到 F，直接輸出。
輸出：AB&&C||EF
stack：||, !, (, >

讀到)，開始 pop 直到遇到 (。所以先輸出 >，再把 (丟掉。
輸出：AB&&C||EF>
stack：||, !

掃描結束後，把 stack 全部 pop 出來，先 pop !，再 pop ||。
最後 postfix 為

AB&&C||EF>!||

若寫成有空格、考卷較清楚的形式，就是

AB&&C||EF>!||

接著做 b 小題，用 postfix 求值。
postfix 是

AB&&C||EF>!||

已知

A=True,B=True,C=False,E=3,F=4

現在用值 stack 逐步計算：

讀到 A，push True
stack：[True]

讀到 B，push True
stack：[True, True]

讀到 &&，pop 兩個值：True && True = True，再 push 回去
stack：[True]

讀到 C，push False
stack：[True, False]

讀到 ||，pop 兩個值：True || False = True，再 push 回去
stack：[True]

讀到 E，push 3
stack：[True, 3]

讀到 F，push 4
stack：[True, 3, 4]

讀到 >，pop 兩個值：3 > 4 = False，再 push 回去
stack：[True, False]

讀到 !，pop 一個值：!False = True，再 push 回去
stack：[True, True]

讀到最後一個 ||，pop 兩個值：True || True = True，再 push 回去
stack：[True]

所以最後運算結果為

True

再做 c 小題，求 prefix。
prefix 可由右往左掃描中序式來完成。原式

A&&B||C||!(E>F)

其實依照結構就是

((A&&B)||C)||(!(E>F))

因此先把子式寫成 prefix：

A && B 的 prefix 是

&&AB

(A && B) || C 的 prefix 是

||&&ABC

E > F 的 prefix 是

>EF

!(E > F) 的 prefix 是

!>EF

最後整個式子的 prefix 是

||||&&ABC!>EF

所以答案為

||||&&ABC!>EF

若老師要你寫成緊湊形式，也可寫成

||||ABC!>EF

但考卷上建議保留空格，比較不會被扣分。

最後整理本題答案：

a. postfix expression

$$\boxed{A\ B\ \&\&\ C\ ||\ E\ F\ >!\ ||}$$

b. 代入 $A = True,\ B = True,\ C = False,\ E = 3,\ F = 4$ 後，用 postfix 計算結果為

$$\boxed{True}$$

c. prefix expression

$$\boxed{||\ ||\ \&\&\ A\ B\ C\ !\ >\ E\ F}$$

5(30 pts) Please point the error and correct it.

a. Functional decomposition is addressed before a system has been decomposed into objects in the object-orientated decomposition.
b. Testing and debugging a program can become more easier because of data encapsulation.
c. Data encapsulation is to separate the specification of a data object and its implementation.
d. According to the time complexities $O(f_1)$ and $O(f_2)$, we can say the f_1 is faster than f_2, where $f_1 = 4n^3 + n^2$ and $f_2 = n^3 + 1$.
e. The space of a pointer of a data member as well as the associated data object cannot be deleted if the destructor of a user specified class is not defined.
f. A derived class inherits all the non-private members data and functions of the base class including constructor and destructor.
g. If class B inherits from class A, then B IS-A A. This implies that A is more specific than B.
h. We can obtain (b) when we assign x=y according to (a).
i. Here shows the doubly linked list.
j. Please find the error and correct it in the following pseudo code.

2. 題目中文翻譯

5.（30 分）請指出錯誤並加以改正。

a. 在物件導向分解中，在系統被分解成物件之前，會先處理功能分解。
b. 因為資料封裝，程式的測試與除錯會變得更容易。
c. 資料封裝就是將資料物件的規格與其實作分離。
d. 根據時間複雜度 $O(f_1)$ 與 $O(f_2)$，其中 $f_1 = 4n^3 + n^2$、$f_2 = n^3 + 1$，我們可以說 f_1 比 f_2 快。
e. 如果使用者自訂類別沒有定義解構子，那麼資料成員指標所指向的空間以及相關資料物件都無法被刪除。
f. 衍生類別會繼承基底類別所有非 private 的資料成員與函式，包括建構子與解構子。
g. 如果類別 B 繼承自類別 A，則 B IS-A A。這表示 A 比 B 更具體。
h. 依照圖 (a)，當我們指定 x=y 時，就可以得到圖 (b) 的結果。
i. 這裡顯示的是 doubly linked list。
j. 請找出下列虛擬碼中的錯誤並修正。

3. 詳細解答

這一大題是在考資料結構與物件導向概念中的「觀念辨錯」，所以作答

重點不是只說對或錯，而是要指出哪裡錯、為什麼錯、正確敘述應該是什麼。下面逐小題整理成考卷可直接寫的版本。

a. 錯誤在於把 functional decomposition 說成是 object-oriented decomposition 之前的必要步驟。這是不正確的。功能分解是傳統程序導向設計常用的方法，會先把系統拆成多個功能模組；但物件導向設計的核心，是直接從系統中辨認物件、類別、屬性與方法，而不是先做功能分解再做物件分解。
正確改法：In object-oriented decomposition, a system is decomposed into objects rather than being functionally decomposed first.
中文可寫：在物件導向設計中，系統應以物件與類別為中心進行分解，而不是先做功能分解。

b. 這句大致方向是對的，但語意不夠精確。資料封裝的主要作用是把資料與操作資料的方法包在一起，並隱藏內部實作，這樣模組間耦合較低，因此測試與除錯通常較容易。原句的問題在於 “more easier” 文法錯誤，而且理由要說得更精確。
正確改法：Testing and debugging can become easier because encapsulation hides implementation details and reduces coupling.
中文可寫：因為資料封裝能隱藏內部實作並降低模組耦合，所以程式的測試與除錯通常會更容易。

c. 這句把 data encapsulation 和 data abstraction 混在一起。將規格與實作分離，比較接近抽象化或 ADT 的觀念；封裝則是把資料與操作資料的函式包在一起，並控制外部存取。
正確改法：Data encapsulation means bundling data and the operations on that data together and restricting direct access to some components.
中文可寫：資料封裝是把資料與操作資料的方法包裝在同一個類別中，並透過存取控制隱藏內部細節。若說明規格與實作分離，較偏向資料抽象化。

d. 錯誤在於只看 Big-O 就說 f_1 一定比 f_2 快。
因為

$$f_1 = 4n^3 + n^2, \quad f_2 = n^3 + 1$$

兩者的階數同為三次，所以

$$O(f_1) = O(n^3), \quad O(f_2) = O(n^3)$$

Big-O 只能說它們屬於同一階的成長率，不能單憑這個就說 f_1 比 f_2 快。事實上若看主項係數，$4n^3$ 比 n^3 大，通常 f_1 還會比較慢。
正確改法：Since both f_1 and f_2 are $O(n^3)$, we cannot say f_1 is faster than f_2 based on Big-O alone.
中文可寫：因為兩者同屬 $O(n^3)$，所以不能只根據 Big-O 說 f_1 比 f_2 快；若比較實際值，f_1 反而通常較大。

e. 這句在考解構子與動態記憶體。若類別成員中有指標並用 new 配置空間，而類別沒有自己定義解構子來 delete，則「指標變數本身」仍會隨物件銷毀而消失，但「指標所指向的動態配置空間」不會自動釋放，造成 memory leak。原句錯在說 pointer 本身也不能被刪除。
正確改法：If a user-defined class has pointer data members and no proper destructor is defined, the dynamically allocated object pointed to by the pointer may not be deleted, causing memory leaks.
中文可寫：若自訂類別中的指標成員指向動態配置的記憶體，而未定義適當解構子，則指標所指向的動態空間不會被自動釋放，會造成記憶體洩漏；但指標成員本身仍會隨物件消失。

f. 錯誤在於說衍生類別會繼承 constructor 與 destructor。建構子與解構子不會被繼承，但會在建立與銷毀物件時依序被呼叫。衍生類別可以繼承 base class 的非 private 成員，但不能說連 constructor/destructor 也繼承。
正確改法：A derived class inherits accessible members of the base class, but constructors and destructors are not inherited.
中文可寫：衍生類別會繼承基底類別可存取的成員，但建構子與解構子不會被繼承。

g. 前半句 “B IS-A A” 是對的，因為 B 繼承 A，表示 B 是 A 的一種。但後半句錯了。應該是 B 比 A 更具體，A 比 B 更一般化。
正確改法：If class B inherits from class A, then B is-a A, which implies that B is more specific than A.
中文可寫：若 B 繼承自 A，表示 B 是 A 的一種，因此 B 比 A 更具體，而 A 比 B 更一般。

h. 這題圖 (a) 表示 x 指向節點 a，y 指向節點 b。圖 (b) 表示 x 與 y 都指向 b。這確實是 x = y 指標指定後的結果，因為這只是把 y 儲存的位址複製

給 x，不會複製節點內容，也不會把 a 改成 b。若題目要你「point the error」，則常見錯誤觀念是把 x=y 誤解成資料複製。
正確說明：After x = y, both x and y point to the same node; the original node pointed to by x may become inaccessible if no other pointer refers to it.
中文可寫：執行 x=y 後，x 會改成和 y 指向同一個節點，因此圖 (b) 是正確結果；但這是位址指定，不是把節點內容複製過去。若原本 x 指向的節點沒有其他指標參照，還可能造成遺失參考。

i. 題目圖中的結構若標成 doubly linked list，需要每個節點有左右兩個鏈結欄位，而且前後指向都要一致。圖上錯誤通常在於 header node 與各節點的左右鏈結方向沒有完整對應，或畫法比較像 circular doubly linked list 卻只稱作 doubly linked list，不夠精確。
正確改法可寫：This should be a circular doubly linked list with a header node, and each node must have both left-link and right-link correctly connected.
中文可寫：此圖應更精確地說是帶有 header node 的 circular doubly linked list；每個節點都應有前驅與後繼指標，且雙向連結必須一致。

j. 這題的虛擬碼錯誤很多，主要是 template 與 friend class 的寫法不完整，ChainIterator、ChainNode、Chain 都應寫成模板類別，而且 operator->() 的回傳型態也錯。若 current->data 是一個 T 物件，則 operator->() 應回傳 T*，也就是 &current->data，不能直接回傳 current->data。此外比較運算子的參數也應寫成 const ChainIterator<T>& right 之類較完整的形式。
可改成如下較正確版本：

```
template <class T>
class Chain;

template <class T>
class ChainNode {
    friend class Chain<T>;
    friend class ChainIterator<T>;
private:
    T data;
    ChainNode<T>* link;
};

template <class T>
class ChainIterator {
public:
```

```
    T* operator->() const { return &current->data; }

    bool operator!=(const ChainIterator<T>& right) const {
        return current != right.current;
    }

    bool operator==(const ChainIterator<T>& right) const {
        return current == right.current;
    }
private:
    ChainNode<T>* current;
};

template <class T>
class Chain {
public:
    Chain() { first = 0; }
private:
    ChainNode<T>* first;
};
```

中文可寫：此段虛擬碼的錯誤在於 template 型別未完整標示、friend class 宣告不完整，以及 operator->() 應回傳資料成員位址而不是資料本身。修正後，ChainNode、ChainIterator、Chain 都應寫成模板類別，並在需要的地方加上 <T>。

若老師要的是精簡版結論，可以最後再補一句：
本題各小題的共同重點，在於分辨物件導向觀念、Big-O 的真正意義、指標與動態記憶體管理、繼承的正確定義，以及 template/iterator 的語法與語意。這些都是資料結構與 C++ 常見的基本考點。

5、(30 pts) Please point the error and correct it.

a. Functional decomposition is addressed before a system has been decomposed into objects in the object-orientated decomposition.

b. Testing and debugging a program can become more easier because of data encapsulation.

c. Data encapsulation is to separate the specification of a data object and its implementation.

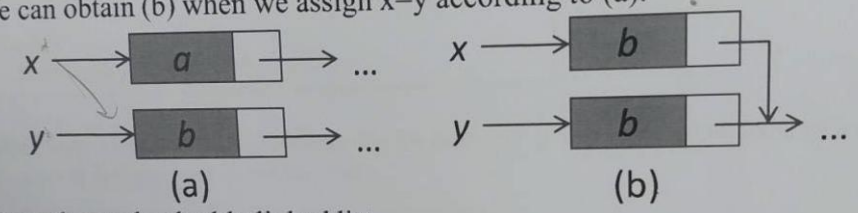
d. According to the time complexities $O(f_1)$ and $O(f_2)$, we can say the f_1 is faster than f_2, where $f_1 = 4n^3 + n^2$ and $f_2 = n^3 + 1$.

e. The space of a pointer of a data member as well as the associated data object cannot be deleted if the destructor of a user specified class is not defined.

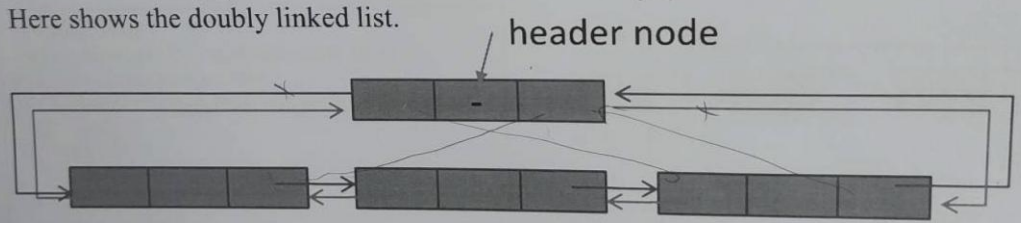
f. A derived class inherits all the non-private members data and functions of the base class including constructor and destructor.

g. If class B inherits from class A, then B IS-A A. This implies that A is more specific than B.

h. We can obtain (b) when we assign x=y according to (a).



i. Here shows the doubly linked list.



j. Please find the error and correct it in the following pseudo code.

```
class ChainIterator {
public:
    T* operator ->() const {return current->data;}

  // equality testing
   bool operator!=(const ChainIterator right) const
      {return current != right.current;}
   bool operator==(const ChainIterator right) const
      {return current == right.current;}
private:
   ChainNode <T>* current; };
```

```
class ChainNode {
   friend class Chain <T>;
   private:
      T data;
      ChainNode <T> *link;
};

class Chain { // definition here
public:
   Chain() {first = 0;}
private:
   ChainNode <T> *first;
};
```

3. 題目中文翻譯

5.（30 分）請指出錯誤並加以改正。

a. 在物件導向分解中，在系統被分解成物件之前，會先處理功能分解。
b. 因為資料封裝，程式的測試與除錯會變得更容易。
c. 資料封裝就是將資料物件的規格與其實作分離。
d. 根據時間複雜度 $O(f_1)$ 與 $O(f_2)$，其中 $f_1 = 4n^3 + n^2$、$f_2 = n^3 + 1$，我們可以說 f_1 比 f_2 快。
e. 如果使用者自訂類別沒有定義解構子，那麼資料成員指標所指向的空間以及相關資料物件都無法被刪除。
f. 衍生類別會繼承基底類別所有非 private 的資料成員與函式，包括建構子與解構子。
g. 如果類別 B 繼承自類別 A，則 B IS-A A。這表示 A 比 B 更具體。
h. 依照圖 (a)，當我們指定 x=y 時，就可以得到圖 (b) 的結果。
i. 這裡顯示的是 doubly linked list。
j. 請找出下列虛擬碼中的錯誤並修正。

4. 詳細解答

這一大題是在考資料結構與物件導向概念中的「觀念辨錯」，所以作答重點不是只說對或錯，而是要指出哪裡錯、為什麼錯、正確敘述應該是什麼。下面逐小題整理成考卷可直接寫的版本。

a. 錯誤在於把 functional decomposition 說成是 object-oriented decomposition 之前的必要步驟。這是不正確的。功能分解是傳統程序導向設計常用的方法，會先把系統拆成多個功能模組；但物件導向設計的核心，是直接從系統中辨認物件、類別、屬性與方法，而不是先做功能分解再做物件分解。
正確改法：In object-oriented decomposition, a system is decomposed into objects rather than being functionally decomposed first.
中文可寫：在物件導向設計中，系統應以物件與類別為中心進行分解，而不是先做功能分解。

b. 這句大致方向是對的，但語意不夠精確。資料封裝的主要作用是把資料與操作資料的方法包在一起，並隱藏內部實作，這樣模組間耦合較低，因此測試與除錯通常較容易。原句的問題在於 “more easier” 文法錯誤，而且理由要說得更精確。
正確改法：Testing and debugging can become easier because encapsulation hides implementation details and reduces coupling.
中文可寫：因為資料封裝能隱藏內部實作並降低模組耦合，所以程式的測試與除錯通常會更容易。

c. 這句把 data encapsulation 和 data abstraction 混在一起。將規格與實作分離，比較接近抽象化或 ADT 的觀念；封裝則是把資料與操作資料的函式包在一起，並控制外部存取。
正確改法：Data encapsulation means bundling data and the operations on that data together and restricting direct access to some components.
中文可寫：資料封裝是把資料與操作資料的方法包裝在同一個類別中，並透過存取控制隱藏內部細節。若說明規格與實作分離，較偏向資料抽象化。

d. 錯誤在於只看 Big-O 就說 f_1 一定比 f_2 快。
因為

$$f_1 = 4n^3 + n^2, \quad f_2 = n^3 + 1$$

兩者的階數同為三次，所以

$$O(f_1) = O(n^3), \quad O(f_2) = O(n^3)$$

Big-O 只能說它們屬於同一階的成長率，不能單憑這個就說 f_1 比 f_2 快。事實上若看主項係數，$4n^3$ 比 n^3 大，通常 f_1 還會比較慢。
正確改法：Since both f_1 and f_2 are $O(n^3)$, we cannot say f_1 is faster than f_2 based on Big-O alone.
中文可寫：因為兩者同屬 $O(n^3)$，所以不能只根據 Big-O 說 f_1 比 f_2 快；若比較實際值，f_1 反而通常較大。

e. 這句在考解構子與動態記憶體。若類別成員中有指標並用 new 配置空間，而類別沒有自己定義解構子來 delete，則「指標變數本身」仍會隨物件銷毀而消失，但「指標所指向的動態配置空間」不會自動釋放，造成 memory leak。原句錯在說 pointer 本身也不能被刪除。
正確改法：If a user-defined class has pointer data members and no proper destructor is defined, the dynamically allocated object pointed to by the pointer may not be deleted, causing memory leaks.
中文可寫：若自訂類別中的指標成員指向動態配置的記憶體，而未定義適當解構子，則指標所指向的動態空間不會被自動釋放，會造成記憶體洩漏；但指標成員本身仍會隨物件消失。

f. 錯誤在於說衍生類別會繼承 constructor 與 destructor。建構子與解構子不會被繼承，但會在建立與銷毀物件時依序被呼叫。衍生類別可以繼承 base class 的非 private 成員，但不能說連 constructor/destructor 也繼承。
正確改法：A derived class inherits accessible members of the base class, but constructors and destructors are not inherited.
中文可寫：衍生類別會繼承基底類別可存取的成員，但建構子與解構子不會被繼承。

g. 前半句 “B IS-A A” 是對的，因為 B 繼承 A，表示 B 是 A 的一種。但後半句錯了。應該是 B 比 A 更具體，A 比 B 更一般化。
正確改法：If class B inherits from class A, then B is-a A, which implies that B is more specific than A.
中文可寫：若 B 繼承自 A，表示 B 是 A 的一種，因此 B 比 A 更具體，而 A 比 B 更一般。

h. 這題圖 (a) 表示 x 指向節點 a，y 指向節點 b。圖 (b) 表示 x 與 y 都指向 b。這確實是 x = y 指標指定後的結果，因為這只是把 y 儲存的位址複製給 x，不會複製節點內容，也不會把 a 改成 b。若題目要你「point the error」，則常見錯誤觀念是把 x=y 誤解成資料複製。
正確說明：After x = y, both x and y point to the same node; the original node pointed to by x may become inaccessible if no other pointer refers to it.
中文可寫：執行 x=y 後，x 會改成和 y 指向同一個節點，因此圖 (b) 是正確結果；但這是位址指定，不是把節點內容複製過去。若原本 x 指向的節點沒有其他指標參照，還可能造成遺失參考。

i. 題目圖中的結構若標成 doubly linked list，需要每個節點有左右兩個鏈結欄位，而且前後指向都要一致。圖上錯誤通常在於 header node 與各節點的左右鏈結方向沒有完整對應，或畫法比較像 circular doubly linked list 卻只稱作 doubly linked list，不夠精確。
正確改法可寫：This should be a circular doubly linked list with a header node, and each node must have both left-link and right-link correctly connected.
中文可寫：此圖應更精確地說是帶有 header node 的 circular doubly linked list；每個節點都應有前驅與後繼指標，且雙向連結必須一致。

j. 這題的虛擬碼錯誤很多，主要是 template 與 friend class 的寫法不完整，ChainIterator、ChainNode、Chain 都應寫成模板類別，而且 operator->() 的回傳型態也錯。若 current->data 是一個 T 物件，則 operator->() 應回傳 T*，也就是 &current->data，不能直接回傳 current->data。此外比較運算子的參數也應寫成 const ChainIterator<T>& right 之類較完整的形式。
可改成如下較正確版本：

```
template <class T>
class Chain;

template <class T>
class ChainNode {
    friend class Chain<T>;
```

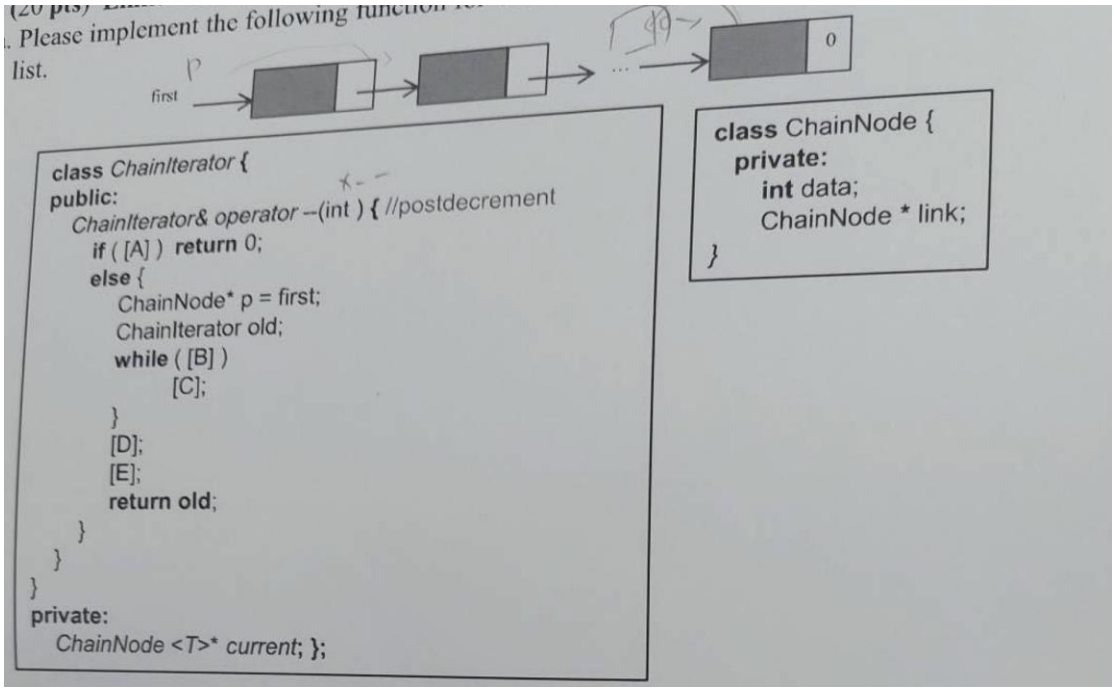
```
    friend class ChainIterator<T>;
private:
    T data;
    ChainNode<T>* link;
};

template <class T>
class ChainIterator {
public:
    T* operator->() const { return &current->data; }

    bool operator!=(const ChainIterator<T>& right) const {
        return current != right.current;
    }

    bool operator==(const ChainIterator<T>& right) const {
        return current == right.current;
    }
private:
    ChainNode<T>* current;
};

template <class T>
class Chain {
public:
    Chain() { first = 0; }
private:
    ChainNode<T>* first;
};
```

中文可寫：此段虛擬碼的錯誤在於 template 型別未完整標示、friend class 宣告不完整，以及 operator->() 應回傳資料成員位址而不是資料本身。修正後，ChainNode、ChainIterator、Chain 都應寫成模板類別，並在需要的地方加上 <T>。

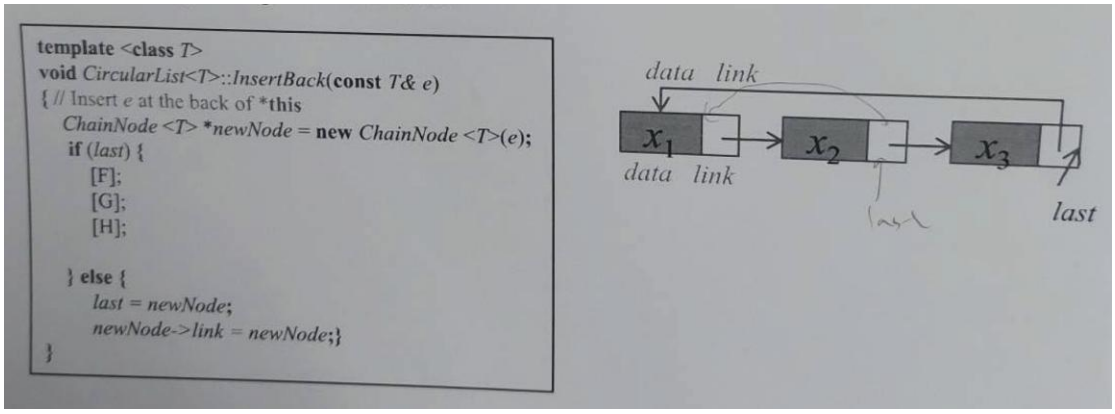
若老師要的是精簡版結論，可以最後再補一句：
本題各小題的共同重點，在於分辨物件導向觀念、Big-O 的真正意義、指標與動態記憶體管理、繼承的正確定義，以及 template/iterator 的語法與語意。這些都是資料結構與 C++ 常見的基本考點。

6. (20 pts) Linked List (you only need to fill the answer in [])

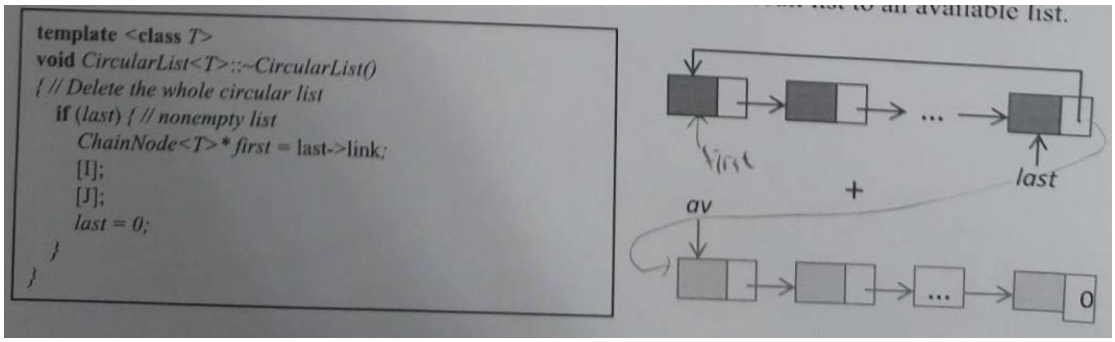
a. Please implement the following function for ChainIterator which is an iterator of a singly linked list.



b. (6 pts) Please complete the following code for inserting a new node to the back of a circle list with a point last pointing to the last node.



c. (4 pts) Please complete the following code for restoring nodes in a circular list to an available list.



2. 中文翻譯

6.（20 分）鏈結串列（只需要填入 [] 中的答案）

a. 請完成單向鏈結串列的迭代器 ChainIterator 中的函式。

b.（6 分）請完成將新節點插入到循環串列尾端的程式碼，其中 last 指向最後一個節點。

c.（4 分）請完成將循環串列中的節點回收到 available list（可用空間串列）的程式碼。

3. 詳細解答（可直接抄寫）

本題重點在於單向串列 iterator 操作、circular list 插入，以及記憶體回收（free list）概念。

(a) iterator postfix --（往前移動到前一個節點）
因為是 singly linked list，沒有前驅指標，所以必須從 first 走訪找到 current 的前一個節點。

[A] current == 0 || current == first
[B] p->link != current
[C] p = p->link

接著更新 iterator：

[D] old.current = current
[E] current = p

完整邏輯說明：若 current 已在第一個節點或為空，無法再往前；否則從 first 開始找前一個節點，最後將 current 往前移，並回傳原位置。

(b) circular list 插入尾端
circular list 中 last->link 會指向 first，因此插入時要同時維護這個關係

[F] newNode->link = last->link
[G] last->link = newNode
[H] last = newNode

完整說明：
新節點的 link 要先指向原本的 first（也就是 last->link），再讓 last 指向新節點，最後更新 last 成為新節點，維持環狀結構不斷裂。

(c) 回收節點到 available list
目的是把整個 circular list 接到 av（free list）上，而不是逐個 delete，提高效率

[I] last->link = av
[J] av = first

完整說明：
first 是整個 circular list 的開頭（last->link），因此只要把最後一個節點接到 av，再讓 av 指向 first，就能把整個環狀串列掛到 available list 上，達到 O(1) 回收效果，而不需要逐一 delete。

總結
(a) 利用走訪找前驅節點（因為 singly list 無 backward pointer）
(b) 插入時維持 circular 結構：新節點連 first，last 指向新節點
(c) 回收時將整個 list 一次接到 av，提高效率

這題核心在考：
iterator 的限制（單向串列不能直接往回）
circular list 的指標維護
memory management（free list 技術）